PYTHON AND JUPYTER ESSENTIALS

This notebook is designed to help you with the fundamentals of Jupyter notebooks and refresh your knowledge of Python in order to give you the best chance of success in the assessment exercises. However if you are already familiar with Jupyter and confident in your Python skills please feel free to skip over this material and make a start on the exercises.

JUPYTER ESSENTIALS

CELLS

This is a markdown cell you can click anywhere on this section of text to edit the contents. Markdown cells allow you to write titles and notes to organise and help explain your code. Editing these cells and looking at this Markdown Cheatsheet will help you understand how you can format them.
Below is code cell containing some Python code, in order to execute this cell clinck anywhere in the cell and then press `Shift` + `Enter`:

PYTHON ESSENTIALS

BASIC DATA TYPES

- integers (`int`)
    - whole numbers, such as `42`, `3`, `56409`
- floats (`float`)
    - decimal numbers, such as `57.346`, `0.2`, `5.0`
- strings (`str`)
    - sequences of characters, such as `Hello, World.`, `'I\'m 37'`
- booleans (`bool`)
    - can only be either `True` or `False`

VARIABLES

A variable is simply a name for something that we need to remember.
We can assign values to variables, which are stored in the computer's memory:

```
age = 25
weight_in_kg = 61.4
name = 'Mary'
is_male = False
```

Python automatically determines the type of the value being assigned to a variable:

```
type(weight_in_kg)
```

Variable names cannot have spaces in them; _underscores_ are typically used between words

MATHS OPERATORS

ADDITION AND SUBTRACTION

```
4 + 4 - 2
```

```
1 - 3 + 0.5
```

`int` and `float` values can be used concurrently.

MULTIPLICATION

```
4 * 2 * 2.5
```

```
3 ** 3
```

`**` is used for exponentiation (*'to the power of'*)

DIVISION

```
4 / 4
```

Division using `/` always results in a value of type `float`.

```
4 / (2 * 3)
```

`(` Parentheses `)` can be used as expected.

OTHER TYPES OF DIVISION

```
8 // 6
```

`//` is used for floor division: the remainder is discarded.

```
365 // 7
```

```
8 % 6
```

`%` is used for what is known as the modulo operation: the remainder is the result.

How many complete hours in a given number of minutes?

```
505 // 60
```

... and how many minutes are left over?

```
505 % 60
```

USE WITH STRINGS

These operators can sometimes be used on strings:

```
5 * 'ab'
```

… but sometimes not:

```
'ab' / 2
```

USING OPERATORS WITH VARIABLES

Operators work with variables as they would if used directly with the assigned values:

```
production = 250
defects = 6

defect_rate = (defects / production) * 100
defect_rate
```

```
item_price = 5
total_cost = 1000

revenue = item_price * (production - defects)
profit = revenue - total_cost
profit
```

ASSIGNMENT OPERATORS

When performing certain operations on a single variable, we can use the following more succinct syntax:

```
attendance = 100
attendance += 3 #equivalent to attendance = attendance + 3
attendance
```

```
refunds = 5
attendance -= refunds
attendance
```

COMMENTS

We can add comments to our code in the following ways:

```
'''
Code for calculation of revenue.
Assumes no failed payments or price changes.
'''
ticket_price = 10
revenue = attendance * ticket_price #assuming refunds are paid in full
revenue
```

- Multi-line comments sit within '''three speech marks'''
- Single line comments (which can share a line with working code) sit to the right of a # character

STRING INDEXING

We can access elements within certain data types (including the characters within strings) by their position, using indexing.
We can access a character, section, or series of characters om a string by providing numerical values (indexes) within `[` square brackets `]`.

```
name = 'Alison'
name[0]
```

If a single value is provided, the character at that specific position is accessed.

```
name[0:3]
```

If a pair of values separated by a `:` are provided, these are interpreted as `start` and `stop` values.

- Note how the character at the `stop` position is not included.
- The difference in values will be the number of characters returned

```
name[:3]
```

If two colons are used, any value to the right of the second colon is interpreted as the `step` value.

```
volcano = 'Eyjafjallajökull'
volcano[0:12:3]
```

```
volcano[3::3]
```

The values (or lack of) are interpreted as `start:stop:step` values (or lack of) respectively.

```
name[-3]
```

```
name[:-3]
```

```
name[-3:]
```

Negative values can be used to refer to position in relation to the end of the string.

- The [position], [start:stop], and [start:stop:step] structures continue to work in the same way.

FINDING THE INDEX OF A GIVEN CHARACTER

```
name.index('s')
```

We have used the `.index()` method to find the index (position) of the first occurrence of the given character.

- Methods and functions will be explained in more detail further on.

TYPE CONVERSION

Attempting operations with values of the wrong type can result in errors:

```
meals = '4'
meals += 1
```

... or mistakes:

```
cost = meals * 8
cost
```

There are several built-in functions we can use to convert the type of a value where possible, each corresponding to the data types we saw earlier:

```
int() float() str() bool()
```

```
meals = int(meals)
meals += 1
meals
```

```
float(meals)
```

Note that conversion of a `float` to an `int` using built-in function `int()` simply truncates the value rather than rounding it:

```
price = 5.99
pounds = int(price)
pounds
```

In this instance, we might instead want to use another built-in function `round()` ...

```
int(round(price, 0)) # The zero here sets the number of decimal places
```

STRING FORMATTING

Python knows that something is a `string` when you put it within either \" or \' marks.

- The same type of quotation mark must be used at the start and finish
- The type of quotation mark not used at the ends can be used within the string

```
"The first man said, 'I'm 50 years old today!'"
```

We can insert variables into strings as follows:

```
name = 'Paul'
age = 40
f"{name} said, 'I'm {age} years old today!'"
```

- We have put an `f` before the string and the required variables within `{braces}`

You may also see the following syntax:

```
"{} said, 'I'm {} years old today!'".format(name, age)
```

The f-strings functionality we saw previously is more concise, but was only introduced in Python 3.6 (December 2016).
The older `.format()` method continues to be supported.

FUNCTIONS

A function is a named block of code which only runs when it is called. In order to call a function we use its named followed by parantheses:
`my_function()`
You can pass data, known as parameters, into a function. An example is below:

```
def my_function(x):
    return 5 * x

print(my_function(3))
print(my_function(5))
print(my_function(9))
```

RAISING ERRORS

DATA STRUCTURES

The standard built-in data structures in python are:

- list: an editable, ordered collection of values
- dictionary: a collection of `key:value` pairs
- tuple: similar to lists, but not editable
- set: a collection of unique values

Data structures are used to input, process, maintain and retrieve data.

- You are likely to encounter lists and dictionaries more frequently than tuples and sets
- You will use other, more specialised data structures in future when using packages such as pandas

LISTS

- lists have a variable number of elements
- elements can be added, removed or modified
- elements are accessed using a zero-based index
- elements do not have to be the same data type

LIST INDEXING

The syntax we can use for accessing list elements by position are the same as we have seen previously for strings, with the number of numerical values and colons determining the interpretation:

- `[position]`
- `[start:stop]`
- `[start:stop:step]`

Again, indexing is zero-based, and negative values can be used to access by position from the end of the list.

LIST INDEXING

```
rain = [6.5, 0, 0, 1.2, 2.6, 1.9, 5.4]
```

```
print(rain[3])
print(rain[3:])
print(rain[:3])
```

LIST INDEXING

```
people = ['Anna', 'Ben', 'Cynthia', 'Dennis', 'Evandro', 'Farhad']
```

```
print(people[::2])
print(people[::-2])
```

HOW ELSE CAN WE WORK WITH LISTS?

- `.append(object)`
- `.pop()`
- `.insert(index, object)`
- `.remove(object)`
- `.extend(list)`
- `.reverse()`
- `.sort()`
- `.copy()`

```
scores = [10, 25, 14]
scores[0] = 50
scores.append('Unknown')
scores
```

- .append() allows us to add a single item to a list

```
scores.pop()
```

```
scores
```

- .pop() returns the last item in the list, and removes it from the list.

```
more_scores = [77, 33, 12]
scores.extend(more_scores)
scores
```

`.extend()` allows us to extend a list with values from another list

```
scores.reverse()
scores
```

```
scores.sort()
scores
```

- .reverse() and .sort() work as expected
- using .sort() on a list of strings will sort them alphabetically

The built-in `len()` function can be used on lists and strings to determine their length:

```
print(scores)
len(scores)
```

```
name = "Helena Bonham Carter"
len(name)
```

DICTIONARIES

- Dictionaries map values to keys
- values are accessed using the key (rather than index)
- values can be modified and further key:value pairs added

```
user = {'Name': 'David',
        'Occupation': 'Magician',
        'Phone': 5554267,
        'Town': 'New York',
       }

user['Town']
```

We can add or modify values in the same way as we would assign or update variables:

```
user['Town'] = 'San Francisco'
user['Gender'] = 'Male'
```

```
user
```

HOW ELSE CAN WE WORK WITH DICTIONARIES?

- `.get(key, default)`
- `.items()`
- `.keys()`
- `.values()`
- `.update(key=value)`
- `.copy()`

```
user.update({'Town': 'Washington',
             'Country': 'USA'})
user
```

- .update() allows us to update an existing dictionary using another dictionary of key:value pairs.

```
user['Age']
```

```
user.get('Age', 'Not provided')
```

- .get() allows us to provide a fallback value should the given key not be found in the dictionary

```
user.keys()
```

```
user.values()
```

`.keys()` and `.values()` return a *view* of the keys and values respectively; we can use the built-in `list()` function to return the list itself from each:

```
list(user.values())
```

TUPLES

- tuples contain an ordered collection of elements
- the number of elements is fixed
- individual elements cannot be modified

```
address = ('Buckingham Palace', 'London', 'SW1A 1AA')
name, area, postcode = address
area
```

Here we have unpacked the tuple, assigning the constituent values to several variables at the same time.

TUPLES VS LISTS

In practice you can usually achieve the same functionality by using a `list`, but:

- using a tuple conveys to others reading the code that the structure of the object is important, and that each element is likely to represent something of different nature
    - it may be important that there are a specific number of elements
    - each element has distinct features, e.g. `name`, `area`, `postcode`

TUPLES VS LISTS

- using a list conveys to others that order is important, elements are more likely to be homogenous, and the number of elements may change
    - each element of our `scores` earlier represents something of the same nature
    - we may not have known how many `scores` there would eventually be in the list

SETS

- All values in a set are unique
- Elements are not ordered or indexed

```
friends = {'Ellie', 'Sarah', 'Amar'}
friends.update(['James', 'Ellie'])
friends.discard('Sarah')
friends
```

Notice how the original order is not retained. Although here the items are displayed alphabetically, the order in which set items are stored in memory should be considered arbitary.

```
required = ['F', 'A', 'C', 'B', 'A', 'C', 'A', 'D', 'H', 'B']
set(required)
```

- The built-in `set()` function is useful for identifying the unique values in a list

```
list(set(required))
```

```
list(sorted(set(required)))
```

- We can use the built-in `list()` function to return a list
- As mentioned previously, the item order of a set is arbitary
  ... but can be resolved using the built-in `sorted()` function

We can check for membership of a set as follows:

```
'Sarah' in friends
```

The same `in` keyword can also be used to check for the presence of a value in a list:

```
0 in rain
```